JavaScript Interview Prep

For Vishal Jat | MERN Stack Fresher / Internship Prep | Simple English Answers

---

Section 1: JavaScript Fundamentals

Q1. What is a closure? Give a simple example.

A closure is when a function remembers the variables from the place it was created, even after that outer function has finished running.

This is useful for making private variables and for things like counters or caching.

```
function makeCounter() {
  let count = 0;
  return function() {
    count = count + 1;
    return count;
  };
}

const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

Tip: In the interview, don't just say the definition. Say 'here is where I used it in my project' if you can.

Q2. What is the Event Loop? Explain call stack, microtask queue, and macrotask queue.

JavaScript can only do one thing at a time. This is called single threaded.

The call stack runs your normal code line by line.

When you use something like setTimeout, it goes to the macrotask queue (also called task queue).

When you use a Promise (.then), it goes to the microtask queue.

The rule is: JavaScript finishes ALL the code in the call stack first. Then it finishes ALL microtasks. Then it takes ONE task from the macrotask queue.

```
console.log('1');

setTimeout(() => {
  console.log('2');
}, 0);

Promise.resolve().then(() => {
  console.log('3');
});

console.log('4');

// Output: 1  4  3  2
```

Tip: Interviewers love giving code like this and asking you to guess the output order. Practice this pattern a lot.

Q3. What is the difference between var, let, and const?

var: old way. It is function-scoped, not block-scoped. It gets 'hoisted' and starts as undefined.

let: block-scoped. You can change its value later, but not use it before it is declared.

const: block-scoped. You cannot change its value after you give it once.

The time between the start of the block and the actual line where let/const is declared is called the Temporal Dead Zone (TDZ). If you try to use the variable during this time, you get an error.

```
console.log(a); // undefined (var is hoisted)
var a = 5;

console.log(b); // ERROR (TDZ)
let b = 10;
```

Q4. What is prototypal inheritance? What is the difference between __proto__ and prototype?

In JavaScript, every object can have a link to another object. This link is used to find properties or methods that are not directly on the object. This chain of links is called the prototype chain.

prototype is a property that exists on functions (used to build objects with 'new').

__proto__ is the actual link on an object that points to its prototype.

```
function Person(name) {
  this.name = name;
}
Person.prototype.sayHi = function() {
  console.log('Hi, I am ' + this.name);
};

const p1 = new Person('Vishal');
p1.sayHi(); // Hi, I am Vishal
```

Q5. Explain 'this' keyword in different situations.

In a normal function, 'this' depends on how the function is called (who is calling it).

In an arrow function, 'this' is NOT its own. It simply uses 'this' from the surrounding (outer) code.

In an object method, 'this' refers to the object before the dot.

This is one of the most common places freshers make mistakes, so practice it well.

```
const obj = {
  name: 'Vishal',
  normalFn: function() {
    console.log(this.name); // 'Vishal'
  },
  arrowFn: () => {
    console.log(this.name); // undefined, arrow does not bind its own 'this'
  }
};
```

Q6. What are Promises? Explain Promise.all, Promise.race, and Promise.allSettled.

A Promise is an object that represents a value which will be available now, later, or never (if there is an error).

Promise.all: waits for ALL promises to finish. If even one fails, the whole thing fails.

Promise.race: gives the result of whichever promise finishes FIRST (success or fail).

Promise.allSettled: waits for all promises, and tells you the result of each one, whether it passed or failed.

```
async function getData() {
  try {
    const res = await fetch('/api/data');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log('Error:', err);
  }
}
```

Q7. What is debouncing and throttling? Can you write the code?

Debounce: wait until the user STOPS doing an action for some time, then run the function. Good for search boxes.

Throttle: run the function only once every fixed amount of time, no matter how many times the event happens. Good for scroll or resize events.

```
// Debounce
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

// Throttle
function throttle(fn, limit) {
  let waiting = false;
  return function(...args) {
    if (!waiting) {
      fn(...args);
      waiting = true;
      setTimeout(() => { waiting = false; }, limit);
    }
  };
}
```

Tip: Practice writing this from memory on plain paper. This is asked as a live coding question very often.

Q8. What is currying? What is function composition?

Currying means changing a function that takes many arguments into a series of functions that each take one argument.

Function composition means combining two or more functions to make a new function, where the output of one becomes the input of the next.

```
// Currying example
function add(a) {
  return function(b) {
    return a + b;
  };
}
add(2)(3); // 5
```

Q9. What is the difference between deep copy and shallow copy? How do you make a deep copy?

Shallow copy: only copies the top level. If the object has another object inside it, that inner object is still shared (linked) between the copy and the original.

Deep copy: copies everything, even the objects inside objects, so nothing is shared.

Ways to deep copy: structuredClone(obj), or JSON.parse(JSON.stringify(obj)) (but this fails on functions and dates), or use a library like lodash cloneDeep.

```
const original = { name: 'Vishal', address: { city: 'Indore' } };
const deepCopy = structuredClone(original);
deepCopy.address.city = 'Bhopal';
console.log(original.address.city); // still 'Indore'
```

Q10. What are higher-order functions? Can you write your own version of map or reduce?

A higher-order function is a function that takes another function as input, or returns a function as output. Examples: map, filter, reduce.

```
Array.prototype.myMap = function(callback) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    result.push(callback(this[i], i, this));
  }
  return result;
};

[1, 2, 3].myMap(x => x * 2); // [2, 4, 6]
```

Tip: Writing your own map/filter/reduce from scratch is a very common 'impress the interviewer' question.

Section 2: More Important JS Questions

Q11. What is the difference between == and ===?

== compares values but converts (changes) the type first if the types are different. This is called type coercion.

=== compares both value AND type, without changing anything. This is called strict equality.

Always prefer === in real code because it avoids confusing bugs.

```
0 == '0'   // true  (string '0' gets converted to number)
0 === '0'  // false (different types)
null == undefined   // true
null === undefined  // false
```

Q12. What is hoisting in JavaScript?

Hoisting means JavaScript moves variable and function declarations to the top of their scope before running the code.

var variables are hoisted and set to undefined.

let and const are hoisted too, but they stay in the Temporal Dead Zone (TDZ) until the line where they are declared, so using them early gives an error.

Normal function declarations are fully hoisted, so you can call them before they appear in the code.

```
sayHi(); // works fine
function sayHi() {
  console.log('Hi');
}
```

Q13. What is the difference between call, apply, and bind?

All three are used to set the value of 'this' manually for a function.

call(): runs the function immediately, and you pass arguments one by one, separated by commas.

apply(): runs the function immediately too, but you pass arguments as a single array.

bind(): does NOT run the function immediately. It returns a new function with 'this' fixed, which you can call later.

```
function greet(city) {
  console.log(this.name + ' from ' + city);
}
const person = { name: 'Vishal' };

greet.call(person, 'Indore');
greet.apply(person, ['Indore']);
const boundFn = greet.bind(person);
boundFn('Indore');
```

Q14. What is event delegation?

Instead of adding a click event to every single child element, you add ONE event listener on the parent element.

When a child is clicked, the click 'bubbles up' to the parent, and you can check which child was actually clicked using event.target.

This is more efficient, especially when items are added or removed dynamically.

Q15. What are the different data types in JavaScript? What is the difference between primitive and reference types?

Primitive types: string, number, boolean, null, undefined, symbol, bigint. These are copied by VALUE.

Reference types: object, array, function. These are copied by REFERENCE (they point to the same place in memory).

This is why changing an object inside a function can affect the original object outside the function, but changing a number does not.

```
let a = 5;
let b = a;
```

```
b = 10;
console.log(a); // 5, not affected

let obj1 = { value: 5 };
let obj2 = obj1;
obj2.value = 10;
console.log(obj1.value); // 10, both point to same object
```

Q16. What is a Promise chain? What is the difference between async/await and .then()?

A promise chain is when you connect multiple .then() calls one after another, where each step waits for the previous one to finish.

async/await is simply a cleaner, easier way to write the same promise logic. It looks like normal step-by-step code, but it still works with promises underneath.

Both do the same job. async/await is usually easier to read, especially with many steps.

```
// Using .then()
fetch('/api').then(res => res.json()).then(data => console.log(data));

// Using async/await (same thing, easier to read)
async function getData() {
  const res = await fetch('/api');
  const data = await res.json();
  console.log(data);
}
```

Q17. What is the difference between null and undefined?

undefined means a variable has been declared, but no value has been given to it yet. JavaScript sets this automatically.

null means 'no value', and it is set on purpose by the programmer to show that a variable should be empty.

Q18. What are pure functions? Why are they useful?

A pure function always gives the same output for the same input, and it does not change anything outside itself (no side effects).

Pure functions are easier to test, debug, and reuse, which is why they are considered good practice.

```
// Pure function
function add(a, b) {
  return a + b;
}

// NOT pure (changes something outside itself)
let total = 0;
function addToTotal(x) {
  total = total + x;
}
```

Q19. What is the difference between synchronous and asynchronous code?

Synchronous code runs line by line, and each line must finish before the next one starts.

Asynchronous code allows some tasks (like API calls, timers, file reads) to run in the background, so the rest of the code does not have to wait for them.

JavaScript uses async code (with callbacks, promises, async/await) so that slow tasks don't freeze the whole program.

Q20. What is destructuring and the spread/rest operator?

Destructuring lets you quickly pull out values from an array or object into separate variables.

The spread operator (...) is used to expand/spread an array or object into individual items.

The rest operator (...) is used to collect many values into a single array (it looks the same as spread, but it works in the opposite direction).

```
const user = { name: 'Vishal', age: 22 };
const { name, age } = user; // destructuring

const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // spread

function sum(...numbers) { // rest
  return numbers.reduce((a, b) => a + b, 0);
}
```

Section 3: How to Stand Out (Simple Tips)

1. Do not just repeat the definition. Try to explain it with a small example or connect it to your own project. Recruiters remember stories, not textbook lines.

2. Practice tracing code out loud. Many companies give you a small code snippet and ask 'what will this print?'. This tests if you really understand the event loop and closures, not just memorized them.

3. Be ready to write small functions on paper or a whiteboard: debounce, throttle, deep clone, your own map/filter, and a simple Promise wrapper.

4. When they ask about React/Node/MongoDB, always try to connect it back to something you actually built. Example: 'I used useMemo in my project to avoid recalculating a big list every time.'

5. It is okay to say 'I am not fully sure, but I think it works like this...' instead of staying silent. Recruiters like to see how you think, not just if you know the exact answer.